UBP - Licenciatura en Inteligencia Artificial

Fundamentos de la Inteligencia Artificial

Material de estudio integrador
Modulos 4 - 5

Volumen II · Representacion del conocimiento, razonamiento logico e introduccion al aprendizaje automatico. El volumen I de este material cubre los Modulos 1 a 3.

Como usar este material

Este **Volumen II** complementa al Volumen I (Modulos 1 a 3) de la asignatura *Fundamentos de la Inteligencia Artificial*. Mientras el primer volumen construye el marco conceptual (que es la IA, agentes y busqueda), este segundo volumen aborda las dos grandes tradiciones que permiten a un agente ir mas alla de la busqueda:

- **Modulo 4 - Representacion del conocimiento y razonamiento logico**: la IA simbolica. El agente deja de actuar por procedimiento y pasa a razonar explicitamente sobre lo que sabe.
- **Modulo 5 - Introduccion al aprendizaje automatico**: la IA basada en datos. El agente ya no necesita que le programemos todo: aprende patrones a partir de ejemplos.

> **Convenciones visuales.** Se mantienen las del Volumen I: naranja para ideas clave, azul para contexto, verde para ejemplos resueltos, rojo para errores frecuentes. Las formulas van en recuadros grises y los pseudocodigos imitan la notacion de Russell & Norvig (AIMA, 4a. ed.).

Indice

Modulo 4 - Representacion del conocimiento y razonamiento logico

- Del agente procedimental al agente basado en conocimiento
- Base de conocimiento, inferencia y nivel de conocimiento
- El mundo del Wumpus
- Fundamentos formales: sintaxis, semantica, implicacion
- Logica proposicional
- Reglas de inferencia, resolucion y clausulas de Horn
- Logica de primer orden (predicados y cuantificadores)

Modulo 5 - Introduccion al aprendizaje automatico

- Que significa aprender a partir de datos
- Tipos de aprendizaje: supervisado, no supervisado, por refuerzo
- Hipotesis, espacio de hipotesis y generalizacion
- Sesgo, varianza, sobreajuste y subajuste
- Arboles de decision y ganancia de informacion
- k-NN: clasificacion y regresion no parametrica
- Metricas de distancia y normalizacion

Sintesis integradora de los 5 modulos

Material de referencia y lecturas recomendadas

Glosario integrador M4-M5

MODULO 4

Representacion del conocimiento y razonamiento logico

De la accion por procedimiento a la deduccion explicita

4.1 - Del agente procedimental al agente basado en conocimiento

Los agentes vistos en los Modulos 2 y 3 saben hacer cosas (percibir, buscar, planificar) pero no saben *por que* hacen lo que hacen. Un agente que busca rutas en un mapa calcula el camino mas corto, pero no "sabe" que una ruta no puede tener longitud negativa, o que dos ciudades distintas no pueden ocupar el mismo lugar. Ese conocimiento abstracto, general y reutilizable es lo que la IA simbolica introduce.

> **Idea central.** Un **agente basado en conocimiento** almacena hechos y reglas sobre el mundo en una **base de conocimiento** (BC), y aplica un mecanismo de **inferencia** para derivar nuevas verdades. No sigue reglas preprogramadas: razona sobre lo que sabe.

El esquema general de un agente de este tipo:

```
funcion AGENTE-BC(percepcion) devuelve una accion
    variables estaticas: BC, una base de conocimiento
                         t,  contador de tiempo, inicializado en 0

    DECIR(BC, CONSTRUIR-SENTENCIA-DE-PERCEPCION(percepcion, t))
    accion := PREGUNTAR(BC, PEDIR-ACCION(t))
    DECIR(BC, CONSTRUIR-SENTENCIA-DE-ACCION(accion, t))
    t := t + 1
    devolver accion
```

Tres operaciones estructuran su funcionamiento:

- **DECIR**: agrega una sentencia a la BC (percepcion o accion tomada).
- **PREGUNTAR**: consulta a la BC que hacer en base al razonamiento sobre lo que sabe.
- **Inferencia**: queda encapsulada dentro de DECIR y PREGUNTAR.

4.2 - Nivel de conocimiento y enfoques

Este diseno permite describir al agente en el **nivel de conocimiento** (lo que sabe y desea), no solo en el nivel de implementacion (como esta programado). Dos enfoques para construirlo:

Enfoque	Descripcion	Ejemplo
Declarativo	Se le "dice" al agente lo que necesita saber mediante sentencias logicas.	"Si hay brillo, entonces hay oro".
Procedimental	El conocimiento se incorpora implicitamente en el codigo del programa.	Rutinas hard-codeadas.

En la practica se combinan: el conocimiento declarativo se compila luego a procedimientos eficientes, y los agentes avanzados integran ambos niveles. Un agente basado en conocimiento tambien puede **aprender**: generar nuevas afirmaciones generales a partir de percepciones individuales.

4.3 - El mundo del Wumpus

El **mundo del Wumpus** (Russell & Norvig, 2021, p. 211) es el entorno canonico para estudiar razonamiento logico. Es una cueva de 4x4 habitaciones con oro, pozos mortales y un monstruo (el Wumpus). El agente debe recoger el oro y salir vivo.

Hedor		Brisa	**PIT**
Brisa Hedor Oro	**PIT**	Brisa	
Hedor		Brisa	
INICIO	Brisa	**PIT**	Brisa

Figura 4.1 - Mundo del Wumpus 4x4 tipico. Hedor en celdas adyacentes al Wumpus; Brisa en celdas adyacentes a un pozo; Brillo donde esta el oro.

REAS del Wumpus:

Elemento	Descripcion
Rendimiento	+1000 por salir con el oro; -1000 por morir; -1 por cada accion; -10 por disparar.
Entorno	Cuadricula 4x4, el agente inicia en [1,1] mirando al este. Wumpus, oro y pozos (20%) distribuidos aleatoriamente.
Actuadores	Avanzar, GirarIzquierda, GirarDerecha, Agarrar, Disparar (una sola flecha), Escalar (solo desde [1,1]).
Sensores	Vector de 5 percepciones: Hedor, Brisa, Brillo, Choque, Grito.

Clasificacion segun el Modulo 2: determinista, discreto, estatico, secuencial, **parcialmente observable** (solo percepciones locales). El desafio clave es la ignorancia inicial sobre la configuracion del mundo, que debe superarse mediante razonamiento logico.

Ejemplo de inferencia. El agente esta en [1,1] (segura por ser el inicio). Percibe [Nada, Nada, Nada, Nada, Nada] - no hay brisa ni hedor. Deduce que [1,2] y [2,1] estan OK. Avanza a [2,1]: percibe brisa. Infiere que al menos una de [1,1], [2,2], [3,1] tiene un pozo. Como [1,1] ya es segura, el pozo esta en [2,2] o [3,1]. Vuelve y va a [1,2], donde percibe hedor sin brisa: el Wumpus esta en [1,3] o [2,2]; como no habia hedor en [2,1], el Wumpus esta en [1,3]. La ausencia de brisa en [1,2] implica que [2,2] no tiene pozo, por lo que el pozo esta en [3,1]. El agente avanza con seguridad a [2,2] y [2,3], donde encuentra el oro.

> **Lo que mostro el Wumpus.** El agente infiere informacion no observable combinando percepciones de distintos momentos y lugares, apoyandose incluso en la *ausencia* de ciertas percepciones. Esta capacidad de razonar con informacion incompleta es imposible con reglas condicion-accion.

4.4 - Fundamentos formales del razonamiento logico

Tres componentes definen cualquier sistema logico:

Componente	Rol	Pregunta que responde
Sintaxis	Reglas que definen las sentencias bien formadas.	Como se escribe correctamente una sentencia?
Semantica	Significado de las sentencias: cuando son verdaderas o falsas con respecto a un modelo.	Que significa la sentencia?
Inferencia	Reglas que permiten derivar nuevas sentencias preservando la verdad.	Que puedo concluir a partir de lo que se?

Modelo e implicacion. Un *modelo* es una asignacion formal de valores de verdad que determina si cada sentencia es verdadera o falsa. La **implicacion logica** se denota:

```
alpha |= beta   si y solo si   M(alpha) subset M(beta)
```

es decir, beta es verdadera en todos los modelos donde alpha lo es. Si BC |= alpha, decimos que BC *implica* alpha.

Analogia utilisima: la implicacion es la *aguja en el pajar* - afirmar que la aguja esta ahi. La inferencia es el proceso de *encontrarla*. Un algoritmo de inferencia `i` es:

- **Solido**: nunca deriva algo que no sea una consecuencia logica (si BC |-i alpha entonces BC |= alpha).
- **Completo**: puede derivar todas las consecuencias logicas (si BC |= alpha entonces BC |-i alpha).

4.5 - Logica proposicional

La logica proposicional es el sistema formal mas simple. Sus unidades son **simbolos proposicionales** (P, Q, R, W_{1,3}, B_{2,1}...) que representan hechos verdaderos o falsos. Se componen con cinco **conectivos**:

Simbolo	Nombre	Significado	Ejemplo
NO	Negacion	P es falso	NO W_{1,3}: "no hay Wumpus en [1,3]"
Y	Conjuncion	Ambos verdaderos	W_{1,3} Y P_{3,1}
O	Disyuncion	Al menos uno verdadero	P_{1,2} O P_{2,1}
=>	Implicacion	Si P entonces Q	B_{1,1} => (P_{1,2} O P_{2,1})
<=>	Bicondicional	P si y solo si Q	W_{1,3} <=> NO W_{2,2}

Precedencia: NO > Y > O > => > <=>

Semantica por tablas de verdad. Un *modelo* es una asignacion de verdadero/falso a cada simbolo. Con n simbolos hay 2^n modelos posibles.

P	Q	NO P	P Y Q	P O Q	P => Q	P <=> Q
F	F	V	F	F	V	V
F	V	V	F	V	V	F
V	F	F	F	V	F	F
V	V	F	V	V	V	V

Figura 4.2 - Tablas de verdad de los cinco conectivos. Nota: P => Q es verdadero salvo cuando P es V y Q es F.

Base de conocimiento del Wumpus en logica proposicional

Simbolos para la celda [x,y]: `P_{x,y}` (pozo), `W_{x,y}` (Wumpus), `B_{x,y}` (brisa), `H_{x,y}` (hedor). Sentencias:

- R1: NO P_{1,1}
- R2: B_{1,1} <=> (P_{1,2} O P_{2,1})
- R3: B_{2,1} <=> (P_{1,1} O P_{2,2} O P_{3,1})
- R4: NO B_{1,1} (no hay brisa en celda inicial)
- R5: B_{2,1} (se percibe brisa en [2,1])

A partir de BC = R1 Y R2 Y R3 Y R4 Y R5 se puede deducir, por verificacion de modelos, que no hay pozo en [1,2] ni en [2,1]: BC |= (NO P_{1,2}) Y (NO P_{2,1}).

Verificacion de modelos: algoritmo basico

```
funcion IMPLICACION-EN-TV(BC, alpha) devuelve V o F
    simbolos := lista de simbolos proposicionales en BC y alpha
    devolver COMPROBAR-TV(BC, alpha, simbolos, {})

funcion COMPROBAR-TV(BC, alpha, simbolos, modelo) devuelve V o F
    si VACIA(simbolos) entonces
        si VERDADERO-LP(BC, modelo) entonces
            devolver VERDADERO-LP(alpha, modelo)
        sino
            devolver V   // BC falsa: la implicacion se cumple trivialmente
    sino
        P := PRIMERO(simbolos); resto := RESTO(simbolos)
        devolver COMPROBAR-TV(BC, alpha, resto, modelo U {P=V}) Y
                 COMPROBAR-TV(BC, alpha, resto, modelo U {P=F})
```

Es solido, completo y simple, pero su complejidad temporal es O(2^n). Para dominios grandes la IA recurre a mecanismos de inferencia basados en **prueba de teoremas**.

4.6 - Reglas de inferencia

Las reglas de inferencia derivan conclusiones nuevas directamente desde las sentencias, sin enumerar modelos. Las principales:

Regla	Forma	Significado
Modus Ponens	De alpha => beta y alpha, inferir beta.	Si "si WumpusVivo entonces Disparar" y "WumpusVivo", concluir "Disparar".
Eliminacion de la conjuncion	De alpha Y beta, inferir alpha (o beta).	De (WumpusAdelante Y WumpusVivo), inferir WumpusVivo.
Doble negacion	NO(NO alpha) === alpha	
De Morgan	NO(alpha Y beta) === (NO alpha) O (NO beta)	Dual: NO(alpha O beta) === (NO alpha) Y (NO beta)
Eliminacion del bicondicional	(alpha <=> beta) === (alpha => beta) Y (beta => alpha)	
Contraposicion	(alpha => beta) === (NO beta => NO alpha)	

Prueba completa del Wumpus (no hay pozo en [1,2]):

1. De R2 por eliminacion del bicondicional: R6 = (B_{1,1} => (P_{1,2} O P_{2,1})) Y ((P_{1,2} O P_{2,1}) => B_{1,1}).
2. Por eliminacion de la conjuncion: R7 = ((P_{1,2} O P_{2,1}) => B_{1,1}).
3. Por contraposicion: R8 = (NO B_{1,1} => NO (P_{1,2} O P_{2,1})).
4. Por Modus Ponens con R4: R9 = NO (P_{1,2} O P_{2,1}).
5. Por De Morgan: R10 = NO P_{1,2} Y NO P_{2,1}.

Conclusion: ni [1,2] ni [2,1] contienen un pozo.

4.7 - Resolucion y clausulas de Horn

La **resolucion** es una regla de inferencia que, combinada con cualquier algoritmo de busqueda completo, produce un procedimiento de inferencia completo para toda la logica proposicional. Opera sobre **clausulas** (disyunciones de literales).

Forma normal conjuntiva (CNF). Toda sentencia proposicional es logicamente equivalente a una conjuncion de clausulas. Procedimiento para convertir a CNF:

1. Eliminar <=> mediante la equivalencia bicondicional.
2. Eliminar =>: (alpha => beta) === (NO alpha O beta).
3. "Mover" las negaciones hacia adentro con De Morgan y doble negacion.
4. Distribuir O sobre Y.

Prueba por refutacion. Para demostrar BC |= alpha, se agrega NO alpha a BC y se busca una contradiccion (clausula vacia). Es la base de los demostradores automaticos.

Clausulas definidas y clausulas de Horn

- **Clausula definida**: disyuncion de literales con *exactamente uno* positivo. Ej.: (NO L_{1,1} O NO Brisa O B_{1,1}).
- **Clausula de Horn**: disyuncion con *a lo sumo uno* positivo. Toda clausula definida es de Horn.

Sobre bases de conocimiento con clausulas de Horn se puede aplicar inferencia eficiente mediante dos estrategias:

Encadenamiento hacia adelante (forward chaining)

Razonamiento *dirigido por los datos*. Parte de los hechos conocidos y aplica reglas cuya parte izquierda se cumple, agregando conclusiones hasta alcanzar el objetivo o agotar hechos. Usado en sistemas expertos y diagnostico.

Encadenamiento hacia atras (backward chaining)

Razonamiento *dirigido por metas*. Parte de la conclusion buscada e identifica que reglas podrian justificarla, demostrando recursivamente cada condicion. Es la base de Prolog.

> **Dato historico.** Prolog (1972) implementa directamente estas reglas: su motor es un encadenamiento hacia atras sobre clausulas de Horn con unificacion. Durante los 80 fue el lenguaje elegido por el proyecto japones Quinta Generacion.

4.8 - Monotonia

La logica proposicional (y la de primer orden) es **monotona**: agregar nueva informacion a BC nunca invalida conclusiones previas:

si BC |= alpha entonces (BC Y beta) |= alpha

Esto diferencia a la logica clasica de las logicas no monotonas (default logic) y de los sistemas probabilisticos (bayesianos), donde nueva evidencia puede revisar conclusiones.

4.9 - Logica de primer orden (LPO)

La logica proposicional tiene un limite expresivo: cada proposicion se refiere a un hecho indivisible. Para decir "las celdas adyacentes a pozos son ventosas" en proposicional hay que escribir una regla para cada celda. La **logica de primer orden** (LPO), tambien llamada *logica de predicados*, amplia el lenguaje con tres categorias de simbolos que imitan la estructura del mundo real:

Categoria	Describe	Ejemplos
Objetos (constantes)	Entidades individuales del dominio.	Wumpus, Juan, [2,3], Celda1_2.
Relaciones (predicados)	Propiedades (aridad 1) o relaciones entre objetos (aridad > 1).	Ventosa(x), Adyacente(x, y), Hermano(Ricardo, Juan).
Funciones	Relaciones donde cada entrada tiene exactamente una salida.	Padre(x), PiernaIzquierda(Juan).

Cuantificadores. Permiten hacer afirmaciones sobre conjuntos de objetos sin enumerarlos:

- **Universal** (ForAll x): para todo x se cumple...
  Ej.: `ForAll x Rey(x) => Persona(x)` ("todos los reyes son personas").
- **Existencial** (Exists x): existe al menos un x tal que...
  Ej.: `Exists x Corona(x) Y SobreLaCabeza(x, Juan)` ("el Rey Juan tiene una corona").

Orden de cuantificadores: cuidado.

- `ForAll x Exists y Ama(x, y)` - todo el mundo ama a alguien (cada quien ama a alguien, posiblemente distinto).
- `Exists y ForAll x Ama(x, y)` - hay alguien que es amado por todos.

Dualidad entre cuantificadores (via negacion):

- `ForAll x NO Gusta(x, Brocoli) === NO Exists x Gusta(x, Brocoli)`
- `ForAll x Gusta(x, Helado) === NO Exists x NO Gusta(x, Helado)`

Igualdad: `Padre(Juan) = Enrique`. Negacion: `NO (x = y)`, util para expresar "al menos dos distintos":

Exists x, y [Hermano(x, Ricardo) Y Hermano(y, Ricardo) Y NO (x = y)]

El Wumpus en LPO

En proposicional hacian falta decenas de reglas (una por celda). En LPO basta **un unico axioma** general:

ForAll s Ventoso(s) <=> Exists r Adyacente(r, s) Y Pozo(r)

Para el tiempo, en lugar de duplicar sentencias para cada instante se cuantifica sobre t:

ForAll t, h, r, c, g Percepcion([h, Brisa, r, c, g], t) => Brisa(t)

$$\text{ForAll } t \quad \text{TieneFlecha}(t+1) \iff (\text{TieneFlecha}(t) \text{ Y NO Accion}(\text{Disparar}, t))$$

Esta concision es la gran ventaja de LPO: expresa regularidades generales que la proposicional obliga a enumerar celda por celda.

Interaccion con la base de conocimiento

Se usan las mismas primitivas DECIR / PREGUNTAR, pero ahora las consultas pueden contener variables:

```
DECIR(BC, Rey(Juan))
DECIR(BC, Persona(Ricardo))
DECIR(BC, ForAll x Rey(x) => Persona(x))

PREGUNTAR(BC, Rey(Juan))         // devuelve V
PREGUNTAR(BC, Hermano(x, Juan))  // devuelve {x/Ricardo}, {x/Gerardo}, ...
```

La respuesta a una consulta con variables ya no es solo V/F: es un **conjunto de sustituciones** para las variables (como en bases de datos relacionales o sistemas expertos).

Cierre del Modulo 4. La logica permite al agente razonar explicitamente, justificar sus decisiones y representar regularidades del mundo. Pero exige conocimiento exacto, completo y sin incertidumbre. Ademas, los agentes logicos no aprenden por si mismos: solo razonan sobre lo que les fue dicho. Esta tension abre naturalmente la puerta al Modulo 5.

MODULO 5

Introduccion al aprendizaje automatico

De las reglas predefinidas a los patrones aprendidos a partir de datos

5.1 - Que significa aprender a partir de datos

Hasta aqui, el conocimiento del agente provenia o bien de la busqueda (M3) o bien de la base de conocimiento (M4), ambos definidos por el disenador. Pero muchos desafios actuales - reconocer imagenes, entender lenguaje natural, anticipar comportamientos, recomendar - no pueden resolverse asi: los patrones estan ocultos en grandes volumenes de datos, no en reglas escritas por humanos.

> **Definicion.** Un agente **aprende** cuando mejora su desempeno al observar datos o experiencias e integra esa informacion para ajustar su conducta futura. En IA, al componente que lo hace se lo llama **aprendizaje automatico** (machine learning). El agente recibe datos, construye un modelo y lo utiliza tanto como hipotesis del mundo como mecanismo operativo.

El aprendizaje puede mejorar cualquier componente del diseno de un agente:

- **Reglas de accion** (que hacer ante un estado).
- **Interpretacion de percepciones** (reconocer objetos en imagenes).
- **Modelo de transicion** (como cambia el entorno).
- **Funciones de utilidad** (ajustar preferencias).
- **Calidad de las acciones** (recompensas y castigos).
- **Generador de problemas y critico** (que explorar y como evaluar).

Induccion vs. deduccion. A diferencia del razonamiento logico, el aprendizaje automatico se basa en **induccion**: generaliza a partir de observaciones. Si el sol salio todos los dias, concluye que saldra manana. Esa conclusion puede ser incorrecta: las predicciones son probabilisticas, no garantizadas.

5.2 - Tres tipos de aprendizaje

Tipo	Retroalimentacion	Objetivo	Ejemplo
Supervisado	Pares (entrada, salida) etiquetados	Aprender funcion h: X -> Y	Reconocer "auto" o "peaton" en una foto; estimar precio de una casa
No supervisado	Solo entradas, sin etiquetas	Descubrir estructura o grupos	Agrupacion (clustering): identificar "gatos" en millones de imagenes sin etiquetas
Por refuerzo	Recompensas y castigos a lo largo del tiempo	Aprender politica optima	Agente de ajedrez que recibe +1/-1 al final; vehiculo autonomo

El Modulo 5 se concentra en **aprendizaje supervisado**, la forma mas estudiada y la base de la mayoria de las aplicaciones modernas. Segun el tipo de salida:

- **Clasificacion**: salida discreta (verdadero/falso, auto/peaton, nublado/soleado).
- **Regresion**: salida continua numerica (precio, temperatura, probabilidad).

5.3 - Hipotesis, espacio de hipotesis y generalizacion

Formalmente, el aprendizaje supervisado se plantea asi:

> Dado un conjunto de entrenamiento {(x_1, y_1), ..., (x_N, y_N)} donde cada y_i = f(x_i),
> objetivo: encontrar una funcion h que aproxime la verdadera f.

- **Hipotesis** (h): la funcion aprendida.
- **Espacio de hipotesis** (H): conjunto de todas las funciones candidatas que el algoritmo podria aprender (lineal, polinomica, red neuronal, arbol...).
- **Conjunto de entrenamiento**: ejemplos usados para ajustar h.
- **Conjunto de prueba**: ejemplos no vistos, usados para evaluar si h **generaliza**.

Elegir el espacio de hipotesis correcto es la mitad del problema. Un espacio demasiado restrictivo (solo rectas) no capturara datos curvos; un espacio demasiado flexible (polinomios de grado alto) se ajustara al ruido en lugar del patron.

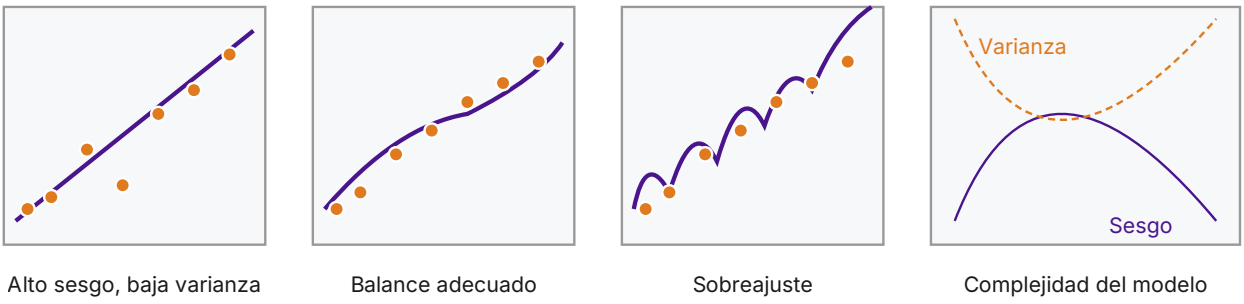


Figura 5.1 - El dilema sesgo-varianza: modelos simples subajustan (alto sesgo); modelos complejos sobreajustan (alta varianza). El optimo esta en el equilibrio.

5.4 - Sesgo, varianza, sobreajuste y subajuste

Concepto	Definicion	Sintoma
Sesgo	Tendencia del modelo a desviarse del valor esperado, por restricciones del espacio H.	Error alto incluso en entrenamiento.
Varianza	Sensibilidad del modelo a pequenas fluctuaciones en los datos.	Modelos muy distintos entre conjuntos de entrenamiento.
Subajuste (underfitting)	El modelo es demasiado simple para capturar los datos.	Mal rendimiento en entrenamiento y en prueba.
Sobreajuste (overfitting)	El modelo se ajusta al ruido del entrenamiento.	Excelente en entrenamiento, pobre en prueba.

Compensacion sesgo-varianza. No existe el modelo perfecto. Modelos rigidos (lineal) tienen alto sesgo y baja varianza; modelos flexibles (polinomios de grado alto, redes grandes) tienen bajo sesgo pero alta varianza. El objetivo es encontrar el **sweet spot** que minimiza el error de generalizacion.

5.5 - Arboles de decision

Un **arbol de decision** representa una funcion que mapea un vector de atributos a una decision mediante una secuencia de pruebas. Nodos internos = atributos; ramas = valores del atributo; hojas = decisiones.

Caso guia: el restaurante. El agente debe decidir si esperar o no una mesa (`WillWait` = V/F) a partir de 10 atributos: Alternativa, Bar, Vie/Sab, Hambriento, Clientes {Ninguno, Algunos, Lleno}, Precio {$, $$, $$$}, Lloviendo, Reserva, Tipo {Frances, Italiano, Tailandes, Hamburgueseria}, Tiempo de espera {0-10, 10-30, 30-60, >60}.

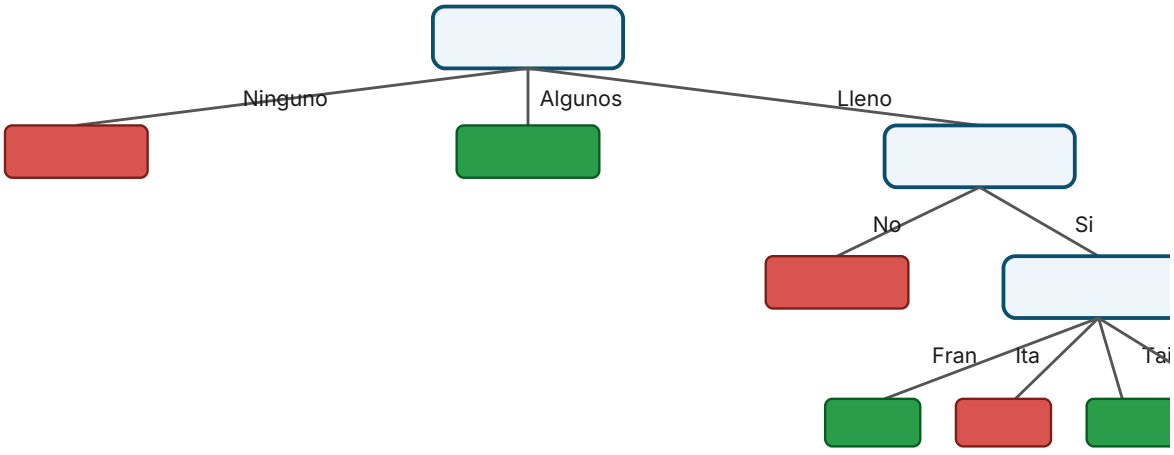


*Figura 5.2 - Arbol de decision aprendido para el problema del restaurante (version simplificada). Nota: el algoritmo descubre que **Clientes** es mas informativo que **Tipo** para la raiz.*

Algoritmo APRENDER-ARBOL-DECISION

Estrategia voraz tipo "divide y venceras": siempre prueba primero el atributo mas *importante* (el que mejor divide los ejemplos), y resuelve recursivamente los subproblemas.

```
funcion APRENDER-ARBOL-DECISION(ejemplos, atributos, ejemplos_padre) devuelve arbol
    si ejemplos esta vacio entonces devolver VALOR-MAYORIA(ejemplos_padre)
    sino si todos los ejemplos tienen la misma clasificacion entonces
        devolver esa clasificacion
    sino si atributos esta vacio entonces devolver VALOR-MAYORIA(ejemplos)
    sino
        A := argmax_{a en atributos} IMPORTANCIA(a, ejemplos)
        arbol := nuevo arbol con nodo raiz A
        para cada valor v de A hacer
            exs := {e en ejemplos : e.A = v}
            subarbol := APRENDER-ARBOL-DECISION(exs, atributos - A, ejemplos)
            anadir rama a arbol con etiqueta (A = v) y subarbol
        devolver arbol
```

Ganancia de informacion y entropia

La **importancia** de un atributo se mide con la **ganancia de informacion**, que a su vez se define a partir de la **entropia** (teoria de la informacion de Shannon):

$$H(V) = - \sum_i P(v_i) \log_2 P(v_i)$$

La entropia mide la incertidumbre de una variable:

- Moneda siempre cara: H = 0 bits (sin incertidumbre).
- Moneda justa: H = 1 bit.
- Dado justo de 4 caras: H = 2 bits.
- Moneda injusta (99% cara): H ~ 0.08 bits.

La **ganancia de informacion** de un atributo es la reduccion esperada de entropia al dividir por ese atributo. Se elige el atributo que la maximiza (reduce mas la incertidumbre).

Curvas de aprendizaje

Para evaluar un modelo se usa una curva que grafica *precision en el conjunto de prueba* vs. *tamano del conjunto de entrenamiento*. Una curva creciente y estable indica buena generalizacion.

5.6 - k-vecinos mas cercanos (k-NN)

k-NN es un algoritmo **no parametrico**: no resume los datos en un numero fijo de parametros, sino que conserva todos los ejemplos de entrenamiento y los consulta directamente para predecir.

> **Idea.** Dado un punto de consulta x_q, buscar los **k ejemplos mas cercanos** en el conjunto de entrenamiento y:
>
> - Para **clasificacion**: votar por la clase mayoritaria entre los k vecinos.
> - Para **regresion**: devolver la media (o mediana, o ajuste lineal) de los valores y de los k vecinos.
>
> En clasificacion binaria se suele elegir k impar para evitar empates.

Efecto de k.

- **k = 1**: muy flexible, pero sensible al ruido - puede sobreajustar.

- **k moderado** (p. ej. 5): mejor balance.
- **k muy grande**: subajuste - predicciones demasiado suaves.

El k optimo se elige con **validacion cruzada** (concepto que se profundiza en Aprendizaje Automatico II).

5.7 - Metricas de distancia

La familia general es la **distancia de Minkowski** (norma L^p):

$$L^p (x_j, x_q) = (\sum_i | x_{j,i} - x_{q,i} |^p)^{1/p}$$

p	Nombre	Cuando usarla
p = 2	Euclidiana	Dimensiones de naturaleza similar (anchura, altura, profundidad)
p = 1	Manhattan	Dimensiones heterogeneas (edad, peso, sexo)
p = inf	Chebyshev	Maxima diferencia en una dimension
-	Hamming	Atributos booleanos: cuenta las posiciones distintas

Normalizacion. Si una dimension se mide en metros y otra en millas, la distancia total dependera arbitrariamente de las unidades. Solucion habitual: estandarizar cada dimension con media mu_i y desviacion sigma_i:

$$x'_{j,i} = (x_{j,i} - \mu_i) / \sigma_i$$

5.8 - Regresion con k-NN

Variantes principales:

1. **Unir los puntos**: interpolar entre los dos vecinos inmediatos. Simple pero muy ruidoso.
2. **Media de k vecinos**: $h(x_q) = (1/k) \sum y_i$. Suaviza pero genera discontinuidades.
3. **Regresion lineal local k-NN**: ajustar una recta a los k vecinos. Mejor captura de tendencias; sigue siendo discontinua.
4. **Regresion ponderada localmente**: cada ejemplo pesa segun su distancia a x_q, con decrecimiento gradual. Resuelve las discontinuidades.

Fortalezas y debilidades de k-NN.

- + Simple, flexible, no asume forma funcional previa.
- + Buenos resultados con datos de baja dimensionalidad.
- - Sensible al ruido y a atributos irrelevantes.
- - Sufre la *maldicion de la dimensionalidad*: en espacios de muchas dimensiones todos los puntos tienden a estar "lejos" entre si.
- - Costoso en inferencia (hay que recorrer el dataset) a menos que se usen estructuras como KD-trees o ball trees.

Sintesis integradora de los 5 modulos

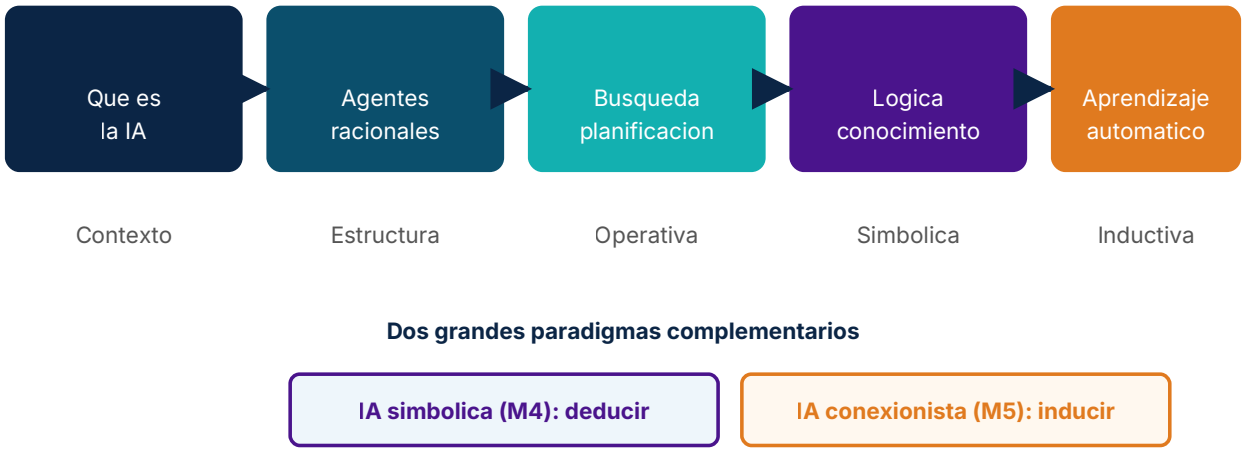


Figura S.1 - El arco completo de los 5 modulos. M4 y M5 encarnan las dos grandes tradiciones de la IA que convergen en los sistemas modernos.

Comparacion logica vs. aprendizaje

Criterio	Logica (M4)	Aprendizaje (M5)
Fuente del conocimiento	El disenador / expertos	Los datos / la experiencia
Tipo de razonamiento	Deductivo (necesariamente cierto)	Inductivo (probabilistico)
Transparencia	Alta: cada paso puede justificarse	Variable (alta en arboles, baja en redes)
Robustez al ruido	Baja: exige hechos exactos	Alta: generaliza desde ejemplos ruidosos
Escalabilidad al dominio	Difusa: axiomas crecen con el dominio	Alta si hay suficientes datos
Capacidad de aprender	Nula sin modulos adicionales	Es su esencia
Ejemplo clasico	Prolog, sistemas expertos (MYCIN)	Deep learning, LLMs, recomendadores

Hacia donde va la disciplina. La IA contemporanea tiende a **sistemas hibridos** que combinan razonamiento simbolico (interpretabilidad, generalizacion, uso de reglas de negocio) con aprendizaje automatico (adaptacion, percepcion, flexibilidad). Es el area denominada *neuro-symbolic AI*, con fuerte presencia en IBM, DeepMind y OpenAI.

Material de referencia y lecturas recomendadas

Para profundizar en el Modulo 4 (razonamiento logico)

Russell, S. & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4a. ed.). Pearson. Capitulos 7 (logica), 8 (LPO), 9 (inferencia en LPO).

Genesereth, M. R. & Nilsson, N. J. (1987). *Logical Foundations of Artificial Intelligence*. Morgan Kaufmann. Clasico sobre fundamentos logicos.

McCarthy, J. (1958). *Programs with Common Sense*. Articulo seminal del Advice Taker; propone al conocimiento declarativo como base de la IA.

Pearl, J. (1988). *Probabilistic Reasoning in Intelligent Systems*. Morgan Kaufmann. Extiende la logica hacia la incertidumbre con redes bayesianas.

Clocksin, W. F. & Mellish, C. S. (2003). *Programming in Prolog* (5a. ed.). Springer. Para experimentar con encadenamiento hacia atras y clausulas de Horn.

Robinson, J. A. (1965). A Machine-Oriented Logic Based on the Resolution Principle. *Journal of the ACM, 12*(1), 23-41. Paper fundacional de la resolucion.

Para profundizar en el Modulo 5 (aprendizaje automatico)

Russell, S. & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4a. ed.). Pearson. Capitulos 19 (aprendizaje supervisado) y 20 (modelos probabilisticos).

Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill. Introduccion clasica aun vigente.

Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer. Enfasis en teoria estadistica.

Hastie, T., Tibshirani, R. & Friedman, J. (2009). *The Elements of Statistical Learning* (2a. ed.). Springer. Disponible gratis en hastie.su.domains/ElemStatLearn.

Geron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow* (3a. ed.). O'Reilly. Practico, con codigo real.

Quinlan, J. R. (1986). Induction of Decision Trees. *Machine Learning, 1*(1), 81-106. Paper original de ID3 (base de los arboles de decision modernos).

Cover, T. & Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Trans. IT, 13*(1), 21-27. Paper original de k-NN.

Recursos interactivos

- SWI-Prolog - interprete gratuito para experimentar con logica declarativa.
- LogicTools - resolvedor interactivo de logica proposicional y LPO.
- scikit-learn examples - galeria de visualizaciones de k-NN, arboles de decision, sesgo-varianza.
- TensorFlow Playground - visualizador en vivo de clasificadores (incluye el dilema sesgo-varianza).
- MLU-Explain (Amazon) - explicadores visuales de los conceptos de M5: sesgo-varianza, arboles, overfitting.
- Dietterich - Decision Tree Tutorial - tutorial detallado con entropia y poda.

Glosario integrador M4 - M5

Modulo 4 - Representacion del conocimiento

Axioma
Sentencia aceptada como verdadera dentro de un sistema logico, base para derivar otras afirmaciones.

Base de conocimiento (BC)
Conjunto de sentencias que describen hechos, reglas y relaciones sobre un dominio en un lenguaje formal.

Clausula
Disyuncion de literales. Unidad de trabajo de la resolucion.

Clausula de Horn
Disyuncion con, como maximo, un literal positivo. Base de Prolog.

Clausula definida
Disyuncion con exactamente un literal positivo.

CNF (Forma Normal Conjuntiva)
Representacion de una sentencia como conjuncion de clausulas.

Completo (algoritmo de inferencia)
Puede derivar toda consecuencia logica de la BC.

Conectivos logicos
Simbolos que construyen sentencias compuestas: NO, Y, O, implicacion, bicondicional.

Cuantificador existencial
"Existe al menos un x tal que P(x)". En este material: Exists x P(x).

Cuantificador universal
"Para todo x, P(x)". En este material: ForAll x P(x).

DECIR / PREGUNTAR
Operaciones estandar para agregar sentencias a la BC y consultarla.

Encadenamiento hacia adelante
Inferencia dirigida por los datos, desde los hechos hacia las conclusiones.

Encadenamiento hacia atras
Inferencia dirigida por metas, desde la conclusion hacia los hechos. Base de Prolog.

Equivalencia logica
Dos sentencias son equivalentes si son verdaderas en los mismos modelos.

Funcion (en LPO)
Relacion donde a cada entrada le corresponde exactamente una salida.

Implicacion logica
alpha |= beta: toda situacion que satisface alpha satisface beta.

Inferencia
Proceso para derivar nuevas sentencias a partir de la BC.

Literal

Sentencia atomica o su negacion.

Logica de primer orden (LPO)
Logica que incorpora objetos, relaciones, funciones y cuantificadores.

Logica proposicional
Sistema formal con proposiciones indivisibles conectadas por operadores logicos.

Modelo
Interpretacion que asigna significado y valores de verdad a las sentencias.

Modus Ponens
Regla de inferencia: de (alpha => beta) y alpha, inferir beta.

Monotonia
Propiedad de la logica clasica: agregar informacion no invalida conclusiones previas.

Mundo del Wumpus
Entorno canonico de 4x4 celdas con oro, pozos y Wumpus; sirve para estudiar razonamiento con informacion incompleta.

Predicado
Simbolo que denota una relacion o propiedad de objetos.

Prueba por refutacion
Metodo que demuestra alpha anadiendo NO alpha a BC y derivando una contradiccion.

Resolucion
Regla de inferencia completa para clausulas; combinada con busqueda produce prueba automatica de teoremas.

Satisfacibilidad
Una sentencia es satisfacible si existe al menos un modelo que la hace verdadera.

Semantica
Rama que estudia el significado y la verdad de las sentencias respecto a los modelos.

Sentencia atomica
Proposicion basica indivisible.

Sintaxis
Reglas que definen las sentencias bien formadas de un lenguaje.

Solido (algoritmo de inferencia)
No deriva nada que no sea una consecuencia logica.

Tautologia
Sentencia verdadera en todos los modelos (equivalente a Verdadero).

Validez
Una sentencia es valida si es verdadera en todos los modelos.

Modulo 5 - Aprendizaje automatico

Aprendizaje automatico (ML)
Capacidad de un sistema para mejorar su desempeno a partir de datos o experiencia.

Aprendizaje no supervisado
El agente aprende patrones en los datos sin retroalimentacion explicita. Ej.: clustering.

Aprendizaje por refuerzo
El agente aprende por recompensas y castigos a lo largo del tiempo.

Aprendizaje supervisado
El agente aprende una funcion a partir de pares (entrada, salida) etiquetados.

Arbol de decision
Modelo que clasifica mediante una secuencia de pruebas sobre atributos.

Atributo
Valor o caracteristica del vector de entrada.

Clasificacion
Tarea supervisada donde la salida es discreta (clases).

Conjunto de entrenamiento
Ejemplos con entrada y salida conocidas usados para ajustar el modelo.

Conjunto de prueba
Ejemplos no vistos durante el entrenamiento, usados para evaluar la generalizacion.

Distancia de Manhattan
Norma L^1: suma de valores absolutos de las diferencias por dimension.

Distancia de Minkowski
Familia general de distancias (L^p); engloba Euclidiana (p=2) y Manhattan (p=1).

Distancia euclidiana
Norma L^2: raiz de la suma de cuadrados de las diferencias.

Entropia
Medida de incertidumbre de una variable aleatoria; base de la ganancia de informacion.

Espacio de hipotesis (H)
Conjunto de todas las funciones candidatas que un algoritmo podria aprender.

Ganancia de informacion
Reduccion esperada de entropia al dividir por un atributo; criterio para elegir atributos en arboles.

Generalizacion
Capacidad de un modelo para predecir correctamente ejemplos no vistos.

Hipotesis (h)
Funcion aprendida por el modelo; aproxima a la funcion verdadera f.

Induccion
Generalizacion a partir de observaciones; opuesto a deduccion.

k-NN (k-vecinos mas cercanos)
Algoritmo no parametrico que predice segun los k ejemplos mas cercanos.

Limite de decision
Superficie que separa las clases en el espacio de atributos.

Modelo no parametrico
Modelo que no resume los datos en un numero fijo de parametros (ej.: k-NN).

Normalizacion
Estandarizar los atributos para que tengan escalas comparables.

Regresion
Tarea supervisada donde la salida es un numero continuo.

Regresion ponderada localmente
k-NN donde cada vecino pesa segun su distancia; evita discontinuidades.

Sesgo
Tendencia del modelo a desviarse del valor esperado por restricciones de H.

Sobreajuste (overfitting)
Modelo demasiado ajustado al ruido del entrenamiento; pierde generalizacion.

Subajuste (underfitting)
Modelo demasiado simple para capturar el patron de los datos.

Validacion cruzada
Tecnica para elegir hiperparametros (como k) evaluando el modelo en multiples particiones.

Varianza
Sensibilidad del modelo a pequenas fluctuaciones en los datos de entrenamiento.

Cierre de la asignatura

Con los Modulos 4 y 5 completamos el panorama fundacional de la Inteligencia Artificial. Hemos recorrido:

1. **Que es la IA** (M1): sus definiciones, enfoques y evolucion historica.
2. **Como se modela** (M2): el agente como unidad conceptual.
3. **Como planifica** (M3): mediante busqueda sistematica o informada.
4. **Como razona** (M4): representando conocimiento y aplicando logica.
5. **Como aprende** (M5): generalizando patrones a partir de datos.

Cada paradigma tiene fortalezas y limites claros. La IA contemporanea no elige uno u otro: los **integra**. Un vehiculo autonomo moderno percibe con redes neuronales profundas (M5), razona sobre reglas de transito con sistemas simbolicos (M4), planifica trayectorias con A* (M3), opera como agente racional en un entorno complejo (M2), y hereda setenta anos de investigacion sobre que significa que una maquina sea inteligente (M1).

> **Que viene despues.** Este material es solo el punto de partida. Las materias siguientes de la carrera profundizan cada una de estas lineas: Aprendizaje Automatico I y II, Redes Neuronales y Deep Learning, Procesamiento de Lenguaje Natural, Vision por Computadora, Aprendizaje por Refuerzo, Etica y Gobernanza de la IA, entre otras.

Volumen II - Material de estudio sintetizado y ampliado por Argentino Gentile a partir del material oficial de la catedra de *Fundamentos de la Inteligencia Artificial*. UBP, Licenciatura en IA. Octubre 2026.